

# exp-fips203-mlkem-kem-encaps-k3

## FIPS 203 Alg.20 ML-KEM-768 KEM Encaps 实现方案

ascendc 工程 (ML-KEM-768 W4b / E20)

2026-07-26

### 摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-kem-encaps-k3/`: 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 20**, 即 ML-KEM-768 ( $k=3$ ) **KEM Encaps**。

**参数锁定**: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem768-parameter-card.md` §3.3 D20: `ek=1184B`, `m=32B`, 输出 `c=1088B` 与 `K=32B`, SIM/生产 **2 launch**。行为基线: 活跃探针 `ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-alg20-kem-encaps-device-k3/`; PKE Encrypt 主体可由本参数组 E14 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-encrypt-k3/` 与 Alg.17 KEM 头组合。**计划 AI Core 数**: Launch 1 为 AIV\_ONLY `blockDim=2` (KEM 头 + Encrypt prep), Launch 2 为 MIX `blockDim=1` (1 AIC + 2 AIV)。**禁止**: 创建 `examples/stable/ml-kem/ml-kem-768/`; 从任何 `frozen/` 目录复制、改写或移植源码; 沿用 k4 的 `c=1568`、`du=11/dv=5`、`re[9]` 或 `pad-to=4/8` 几何。

### 目录

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>工程边界</b>                            | <b>2</b> |
| 1.1      | 允许的代码来源与依赖                             | 2        |
| 1.2      | 禁止项                                    | 2        |
| <b>2</b> | <b>数学语义 (Alg.20 → Alg.17 → Alg.14)</b> | <b>2</b> |
| 2.1      | 输入输出                                   | 2        |
| 2.2      | 阶段公式                                   | 3        |
| <b>3</b> | <b>AscendC 实现规划</b>                    | <b>3</b> |
| 3.1      | Launch 与 AI Core 规模                    | 3        |
| 3.2      | GM 几何                                  | 3        |
| 3.3      | 同步与流水                                  | 4        |
| <b>4</b> | <b>AscendC API 表</b>                   | <b>4</b> |
| 4.1      | 评估过但未采用                                | 5        |
| <b>5</b> | <b>数学步骤覆盖率</b>                         | <b>5</b> |

|                    |          |
|--------------------|----------|
| 目录                 | 2        |
| <b>6 验证计划</b>      | <b>6</b> |
| 6.1 默认验收 . . . . . | 6        |
| 6.2 可选对照 . . . . . | 6        |
| <b>7 产出物</b>       | <b>6</b> |

## 1 工程边界

### 1.1 允许的代码来源与依赖

| 来源                                                                                          | 用途                                                            |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-<br/>alg20-kem-encaps-device-k3/</code> | 活跃 D20 k3 行为基线；可复制后自包含适配到本 exp，最终不得保留对探针目录的编译期 include/import |
| <code>examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-encrypt-k3/</code>        | 活跃 E14 incubating PKE Encrypt；提供已自包含的 k3 Alg.14 主体            |
| <code>examples/incubating/ml-kem/ml-kem-1024/exp-fips203-mlkem-kem-encaps-k4/</code>        | 活跃 k4 incubating KEM Encaps 结构参考；仅用于 retarget 到 k3 锁定字节长与几何   |
| <code>library/shared/</code>                                                                | SHAKE/Keccak、统一整数压缩、公共设备侧头文件等共享库                              |
| CANN / tikicpulib / CAModel                                                                 | 编译、CPU 孪生与 SIM 运行环境                                           |

### 1.2 禁止项

- 禁止从 `ascendc-tests/frozen/` 或 `examples/frozen/` 拿源码、customspec 或路线。
- 禁止把  $h = H(ek)$ 、 $r$ 、 $y$ 、 $e_1$ 、 $e_2$ 、 $\hat{A}$  等中间态作为默认 I/O 落盘；debug dump 只能由显式开关启用。
- 禁止 Host 预填  $r$  冒充设备 KEM 头； $r$  必须由设备侧  $G(m\|H(ek))$  产生并供 Encrypt prep 消费。
- 禁止为  $H/G$  单独增加第三次生产 launch。

## 2 数学语义 (Alg.20 $\rightarrow$ Alg.17 $\rightarrow$ Alg.14)

### 2.1 输入输出

| 张量 / 文件                       | 契约                                                                                     |
|-------------------------------|----------------------------------------------------------------------------------------|
| <code>input/ek_kem.bin</code> | 1184B，等于 PKE public key；前 1152B 为 $\hat{t}$ 的 ByteEncode <sub>12</sub> ，末 32B 为 $\rho$ |

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| input/m.bin                | 32B, Alg.20 internal message; 设备侧 $G(m\ H(ek))$ 的前半为共享秘密 $K$ , 后半为 Encrypt 随机性 $r$                 |
| input/lut_even_stacked.bin | NTT Stage2 LUT; 由本目录 golden 脚本生成; 不                                                                |
| lut_odd_stacked.bin        | 是用户可见交付输入                                                                                          |
| output/c.bin               | 1088B, $c = c_1\ c_2$ , 其中 $c_1 = 960B$ ( $d_u = 10$ , 3 poly), $c_2 = 128B$ ( $d_v = 4$ , 1 poly) |
| output/K.bin               | 32B, $G(m\ H(ek))$ 前半; 由设备侧 KEM 头写出                                                                |

## 2.2 阶段公式

1. KEM 头:  $h = H(ek) = \text{SHA3-256}(ek)$ ,  $(K\|r) = G(m\|h) = \text{SHA3-512}(m\|h)$ 。
2. Alg.14 行 3-7: 取  $\rho = \text{ek}[1152:1184]$ , 对  $(j, p) \in \{0, 1, 2\}^2$  生成  $\hat{A}[j, p]$ , GM 形状锁定为  $[9, 256]$  int32。
3. Alg.14 行 8-15: 用设备产生的  $r$  采样  $y\|e_1\|e_2$  共 7 个 poly; GM 形状锁定为  $[7, 256]$ 。
4. Alg.14 行 16-21: 计算  $\hat{r} = \text{NTT}(y)$ , 得  $\hat{u}$  与  $\hat{tr}$ , 再对  $u_0, u_1, u_2, v$  执行真 **INTT batch=4**。
5. Alg.14 行 22: 按  $d_u = 10, d_v = 4$  pack 得  $c = c_1\|c_2$ ; KEM 头已写出的  $K$  与  $c$  共同作为输出。

## 3 AscendC 实现规划

### 3.1 Launch 与 AI Core 规模

| Launch              | 核类型 /<br>blockDim       | 数据划分与理由                                                                                                                                                         |
|---------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 prep+head         | AIV_ONLY,<br>blockDim=2 | block0 先执行 KEM 头, 产生 $K$ 与 $r$ ; 随后继承 E14/D14 prep: $\hat{A}$ 共 9 poly 按 5+4 分给两个 AIV, $y\ e_1\ e_2$ 共 7 poly 生成到真实 GM。选择 2 AIV 是参数卡 §3.3 D20 对 D14 prep 的锁定继承。 |
| 2 com-<br>pute+pack | MIX,<br>blockDim=1      | 1 AIC + 2 AIV; 复用 E14/D14 compute, $\text{NTT}(y)$ 只处理真实 3 poly, INTT batch4 连续 2+2, tail pack 输出 'c=1088B'。                                                    |

### 3.2 GM 几何

| 对象        | 锁定几何                                                                   |
|-----------|------------------------------------------------------------------------|
| $\hat{A}$ | $[9, 256]$ int32; 9 个真实 poly, 禁止 pad 到 16                              |
| re        | $[7, 256]$ int32; $y[0..2] \    \ e1[0..2] \    \ e2[0]$ , 随机性来自设备 $r$ |

|               |                                                                                |
|---------------|--------------------------------------------------------------------------------|
| $\hat{y}$ NTT | polyvec3; 复用 B5/NTT 真实 poly 几何, 不引入假 poly                                      |
| INTT batch4   | S0 [8,256] int8, Stage2 mat_c [32,128] int32;<br>Cube 的 M 对齐垫只是硬件对齐, 不表示假 poly |
| Tail pack     | $u$ 三 poly 用 $d_u = 10$ pack 为 960B; $v$ 一 poly 用 $d_v = 4$ pack 为 128B        |

### 3.3 同步与流水

KEM 头写出  $K$  与  $r$  后, prep 的 PRF/CBD 必须在同一 launch 内读到完整  $r$ ; 需要用本地已验的 AIV 内同步/顺序约束, 禁止只凭 CPU 顺序假设 SIM 可见。两次 launch 之间由 Host 顺序保证 GM 可见性。Compute 内部沿用 E14/D14 已验同步: AIV 写平面 Stage1/Stage3 GM 后以 CrossCore flag 与 AIC 交接; AIC 写完 mat\_c 后 AIV 等待再做 Stage3 merge、INTT 与 tail pack。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

## 4 AscendC API 表

本实现不引入 E20 独有的新 AscendC 矢量/矩阵 API; KEM 头使用 shared Keccak ‘Sha3OneShot’。下表 API 均已有索引记录, 使用约束按 `library/documents/CANN-AscendC` 算子开发接口参考-查阅索引.md 执行。

| API 名                                | 分类 / 文档     | Atlas A2 | 本用例 dtype        | 备注                                   |
|--------------------------------------|-------------|----------|------------------|--------------------------------------|
| GetBlockIdx / GetSubBlockIdx         | 系统变量 / 资源   | ✓        | 标量索引             | 区分 AIV 分片与 MIX 子核; KEM 头仅固定 block 执行 |
| GlobalTensor / LocalTensor           | 基础结构        | ✓        | uint8/int8/int32 | GM/UB 张量契约必须与本 spec 几何一致             |
| DataCopy                             | Memory 数据搬运 | ✓        | uint8/int8/int32 | 连续块搬运; 长度与地址遵守 32B 对齐约束              |
| Duplicate                            | Memory 矢量计算 | ✓        | uint8/int32      | 用于 UB 清零、pad 与 tail buffer 初始化       |
| PipeBarrier                          | Pipe 同步     | ✓        | —                | MTE/Vector/Cube 阶段交界显式同步             |
| CrossCoreSetFlag / CrossCoreWaitFlag | MIX 同步      | ✓        | —                | AIV/AIC 通过 GM 交接时使用; CPU 过不能删        |

|                     |                      |        |                       |                                                                         |
|---------------------|----------------------|--------|-----------------------|-------------------------------------------------------------------------|
| Mmad / Fixpipe      | 矩阵计算                 | √      | int8*int8->int32      | NTT/INTT<br>Stage2 MMAD;<br>INTT batch4 的逻辑 M=8, 硬件 M 对齐垫到 16           |
| ShiftRight          | 矢量算术                 | √      | int32                 | limb 拆分、<br>Barrett 压缩与<br>ByteEncode 辅助<br>右移                          |
| Muls / Add / Sub    | 矢量算术                 | √      | int32/int16           | limb、模加减、压<br>缩/编码辅助                                                    |
| Cast                | 类型转换                 | √ (受限) | int32/int16/half/int8 | 遵守 A2<br>Cast 链约束; 不<br>得假设存在<br>int32->int8 单<br>跳                     |
| Gather              | 矢量搬运/选择              | √      | int32                 | 仅允许在 Alg.11<br>/ ByteEncode 等<br>非 NTT S1-S3 段<br>按既有路径使用;<br>NTT 三段内禁用 |
| GetValue / SetValue | LocalTensor 标<br>量访问 | √      | uint8/int32           | 仅用于 pack 与<br>KEM 头尾标量缺<br>口; 不得把<br>Compares bit 包<br>当逐 lane 布尔       |

4.1 评估过但未采用

| API / 路线                       | 不采用原因                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------|
| Compares/GatherMask compact 链  | Alg.20 E20 不新增 reject compact; 沿用 E14/D14 已<br>验 pack 路径, 不新增比较掩码风险                   |
| Host 预填 ‘coins.bin’ 作<br>生产随机性 | Alg.20/17 要求 $r = G(m\parallel H(ek))$ ; Host ‘coins’ 只能<br>作为 golden 或调试对照, 不作默认生产输入 |
| 第三 launch KEM head             | 参数卡 §3.3 D20 锁定 SIM 2 launch; KEM 头必须并<br>入 prep launch                               |

5 数学步骤覆盖率

| 数学步骤                              | 实现方式 / API                                  | 覆盖      | 缺口说明                              |
|-----------------------------------|---------------------------------------------|---------|-----------------------------------|
| $H(ek)$ 与 $G(m\ H(ek))$           | shared Keccak<br>SHA3-256/512 +<br>UB/GM 拼接 | 100% 设备 | ‘K’ 与 ‘r’ 均由设备 KEM 头产生            |
| 取 $\rho$ 、采样 $\hat{A}[9]$         | AIV_ONLY +<br>XOF/rej + DataCopy            | 100% 设备 | 5+4 分片；无零垫                        |
| 采样 $y\ e_1\ e_2[7]$               | AIV_ONLY +<br>PRF/CBD                       | 100% 设备 | 真实 7 poly；不借 k4 9 poly            |
| NTT( $y$ ) 与内积                    | MIX AIV/AIC + Mmad                          | 100% 设备 | NTT 内无 Gather                     |
| INTT batch4                       | MIX AIV/AIC +<br>Stage3 merge               | 100% 设备 | 真 polyvec4,<br>AIV 连续 2+2         |
| Compress/ByteEncode<br>$d = 10/4$ | 统一整数压缩 + pack                               | 部分向量    | pack 尾部标量<br>缺口沿用<br>E14/D14 已验实现 |

## 6 验证计划

### 6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-kem-encaps-k3
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望:c.bin 为 1088B,K.bin 为 32B;两者均与 golden 对拍 PASS。SIM 通过后记录 Total tick 到 qa/active\_sim\_regress\_summary.md,并检查用例根目录无 stray core\*.dump、profile\_\*log\*.toml。

### 6.2 可选对照

后续可补 liboqs-768 encaps KAT; E20 本轮门禁为 CPU + SIM\_DIRECT=1 sim。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

## 7 产出物

- exp-fips203-mlkem-kem-encaps-k3-实现方案-customspec.tex
- exp-fips203-mlkem-kem-encaps-k3-实现方案-customspec.pdf
- 后续实现文件须与本 customspec 同目录自包含,且自研 Python/C/AscendC 写详细中文注释。